Atividade prática — Refatoração em Python

10 questões · 6 estratégias · dificuldade variável

Questão 1

Uma aplicação de gerenciamento de tarefas precisa identificar se uma tarefa pode ser iniciada. O código funciona, mas foi escrito por diferentes desenvolvedores e apresenta inconsistências na nomenclatura.

```
None
def start_task(task, user):
    x = task["status"]
    y = user["active"]
    z = task["assigned_to"]

    if x == "Pending" and y and z == user["username"]:
        result = True
    else:
        result = False

    print("Task:", task["title"])
    print("Can start:", result)

    return result
```

"Quais estratégias de refatoração você utilizaria para melhorar esse código? Explique quais problemas identificou e justifique a escolha de cada estratégia."

Questão 2

Um sistema de pagamento precisa calcular o valor que será cobrado de um cliente após a aplicação de uma taxa de serviço.

```
None
def process_payment(order):
    subtotal = order["quantity"] * order["unit_price"]
```

```
    service_fee = subtotal * 0.08

    payment_amount = subtotal + service_fee

    print("Processing payment...")
    print("Amount:", payment_amount)

    return payment_amount
```

"Quais estratégias de refatoração você utilizaria para melhorar esse código? Explique quais problemas identificou e justifique a escolha de cada estratégia."

Questão 3

Uma plataforma de cursos possui uma função responsável por determinar se um estudante pode receber um certificado.

```
None
def issue_certificate(student):
    if student is None:
        return False

    if student["completed_lessons"] < student["total_lessons"]:
        return False

    if student["final_grade"] < 7:
        return False

    if student["payment_status"] != "Paid":
        return False

    print("Certificate issued:", student["name"])
    return True


def has_completed_course(student):
    return (
        student["completed_lessons"] >= student["total_lessons"]
```

```
        and student["final_grade"] >= 7
        and student["payment_status"] == "Paid"
    )
```

"Quais estratégias de refatoração você utilizaria para melhorar esse código? Explique quais problemas identificou e justifique a escolha de cada estratégia."

Questão 4

Um sistema de suporte técnico precisa classificar chamados conforme sua prioridade.

```
None
def classify_ticket(ticket):
    if (
        ticket["severity"] == "Critical"
        and ticket["affected_users"] >= 50
        and ticket["system_status"] == "Unavailable"
        and ticket["created_hour"] >= 18
    ):
        priority = "Emergency"
    elif ticket["severity"] == "High" and
ticket["affected_users"] >= 10:
        priority = "High"
    else:
        priority = "Normal"

    print("Ticket:", ticket["id"])
    print("Priority:", priority)

    return priority
```

"Quais estratégias de refatoração você utilizaria para melhorar esse código? Explique quais problemas identificou e justifique a escolha de cada estratégia."

Questão 5

Uma aplicação de gerenciamento de imagens possui a seguinte estrutura:

```
None
image_manager/
│
├── image.py
├── image_processing.py
└── gallery.py
```

Arquivo `image.py`:

```
None
class Image:
    def __init__(self, filename, width, height, format):
        self.filename = filename
        self.width = width
        self.height = height
        self.format = format

    def resize(self, new_width, new_height):
        self.width = new_width
        self.height = new_height
```

Arquivo `image_processing.py`:

```
None
def calculate_aspect_ratio(image):
    return image.width / image.height


def calculate_file_name(image):
    return image.filename.lower().replace(" ", "_")
```

Arquivo `gallery.py`:

```
None
from image_processing import calculate_aspect_ratio,
calculate_file_name


class Gallery:
```

```
    def __init__(self):
        self.images = []

    def add_image(self, image):
        self.images.append(image)

    def display_image_information(self, image):
        ratio = calculate_aspect_ratio(image)
        filename = calculate_file_name(image)

        print("Filename:", filename)
        print("Dimensions:", image.width, "x", image.height)
        print("Aspect ratio:", ratio)
```

"Quais estratégias de refatoração você utilizaria para melhorar esse código? Explique quais problemas identificou e justifique a escolha de cada estratégia."

Questão 6

Uma aplicação de reservas de hotéis possui a seguinte implementação:

```
None
def calculate_reservation(reservation):
    room_price = reservation["daily_rate"] * reservation["days"]

    discount = calculate_discount(reservation, room_price)

    total = room_price - discount

    print("Guest:", reservation["guest"])
    print("Room:", reservation["room_type"])
    print("Original price:", room_price)
    print("Discount:", discount)
    print("Final price:", total)

    return total
```

```
def calculate_discount(reservation, room_price):
    if reservation["guest_type"] == "Member":
        return room_price * 0.10

    return 0
```

"Quais estratégias de refatoração você utilizaria para melhorar esse código? Explique quais problemas identificou e justifique a escolha de cada estratégia."

Questão 7

Um sistema de gerenciamento de funcionários possui esta estrutura:

```
None
employee_system/
│
├── employee.py
├── payroll.py
└── report.py
```

Arquivo `employee.py`:

```
None
class Employee:
    def __init__(self, name, salary, department):
        self.name = name
        self.salary = salary
        self.department = department
```

Arquivo `payroll.py`:

```
None
def calculate_annual_salary(employee):
    return employee.salary * 12


def calculate_department(employee):
```

```
    return employee.department.upper()
```

Arquivo `report.py`:

```
None
from payroll import calculate_annual_salary, calculate_department


def generate_employee_report(employee):
    annual_salary = calculate_annual_salary(employee)
    department = calculate_department(employee)

    print("Employee:", employee.name)
    print("Department:", department)
    print("Annual salary:", annual_salary)

    return annual_salary
```

"Quais estratégias de refatoração você utilizaria para melhorar esse código? Explique quais problemas identificou e justifique a escolha de cada estratégia."

Questão 8

Um sistema de biblioteca precisa processar devoluções de livros e calcular eventuais multas.

```
None
def process_return(book, user, return_date):
    if book["borrowed"] == False:
        print("Book was not borrowed")
        return False

    overdue_days = max(0, (return_date - book["due_date"]).days)

    if overdue_days > 0:
        fine = overdue_days * 2.50
    else:
        fine = 0
```

```
    book["borrowed"] = False
    book["borrower"] = None

    user["pending_fines"] += fine

    print("Book returned:", book["title"])
    print("Overdue days:", overdue_days)
    print("Fine:", fine)

    return fine
```

"Quais estratégias de refatoração você utilizaria para melhorar esse código? Explique quais problemas identificou e justifique a escolha de cada estratégia."

Questão 9

Uma aplicação de controle financeiro apresenta a seguinte organização:

```
None
finance_app/
│
├── transaction.py
├── finance.py
└── dashboard.py
```

Arquivo `transaction.py`:

```
None
class Transaction:
    def __init__(self, description, amount, category,
transaction_type):
        self.description = description
        self.amount = amount
        self.category = category
        self.transaction_type = transaction_type
```

Arquivo `finance.py`:

```
None
def calculate_balance(transactions):
    balance = 0

    for transaction in transactions:
        if transaction.transaction_type == "Income":
            balance += transaction.amount
        else:
            balance -= transaction.amount

    return balance


def get_transaction_category(transaction):
    return transaction.category.upper()
```

Arquivo `dashboard.py`:

```
None
from finance import calculate_balance, get_transaction_category


def display_dashboard(transactions):
    balance = calculate_balance(transactions)

    print("FINANCIAL DASHBOARD")
    print("-------------------------")
    print("Current balance:", balance)

    for transaction in transactions:
        category = get_transaction_category(transaction)

        print(
            transaction.description,
            "|",
            category,
            "|",
            transaction.amount
        )
```

```
    return balance
```

"Quais estratégias de refatoração você utilizaria para melhorar esse código? Explique quais problemas identificou e justifique a escolha de cada estratégia."

Questão 10

Uma plataforma de comércio eletrônico possui um processo de finalização de pedidos.

None

```
def finalize_order(order, customer):
    customer_data = customer["name"]
    items = order["items"]

    total = 0

    for item in items:
        item_price = item["quantity"] * item["unit_price"]
        total += item_price

    if (
        customer["account_status"] == "Active"
        and customer["purchase_history"] >= 5
        and total >= 500
        and order["payment_status"] == "Approved"
    ):
        discount = total * 0.12
    else:
        discount = 0

    final_amount = total - discount

    print("ORDER SUMMARY")
    print("-------------------------")
    print("Customer:", customer_data)
    print("Items:", len(items))
    print("Subtotal:", total)
    print("Discount:", discount)
```

```
    print("Final amount:", final_amount)

    order["status"] = "Completed"

    return final_amount


def calculate_item_price(item):
    return item["quantity"] * item["unit_price"]


def is_eligible_for_discount(customer, total):
    return (
        customer["account_status"] == "Active"
        and customer["purchase_history"] >= 5
        and total >= 500
        and customer["payment_status"] == "Approved"
    )
```

"Quais estratégias de refatoração você utilizaria para melhorar esse código? Explique quais problemas identificou e justifique a escolha de cada estratégia."